AUTOM-02: Settings API

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 2 of 9 | **Created:** January 2026 | **Last Updated:** 07/31/2026

The Settings API (also called Settings 2.0) is Dynatrace's modern REST API for configuration management. It provides a unified way to manage all Dynatrace settings through JSON objects with schema validation.

Table of Contents

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
API Token	Token with `settings.read`, `settings.write` scopes
Tenant URL	Your Dynatrace SaaS tenant URL
HTTP client	curl, Postman, or similar tool

Learning Objectives

By the end of this notebook, you will:

- Understand the Settings 2.0 API architecture
- Know how to read and write configuration objects
- Be able to list available schemas and their structure
- Handle common API patterns and error cases

---

1. Introduction

Settings API vs Classic Config API

Aspect	Settings API (2.0)	Classic Config API
Format	Unified JSON schemas	Mixed endpoints
Validation	Schema-based	Varies by endpoint
New features	All new features	Legacy only
Recommended	Yes	Deprecated for new configs

Key Concepts

Concept	Description
Schema	Defines structure and validation rules for a config type
Object	An instance of configuration following a schema
Scope	Where the config applies (tenant, host, service, etc.)
Object ID	Unique identifier for a configuration object

Sprint 1.337 (April 2026) Updates

Configuration API → Settings v2 migration accelerated. As of Dynatrace SaaS sprint 1.337, many remaining Configuration API endpoints are now covered by Settings endpoints in Environment API v2. The legacy endpoints stay active for backward compatibility, but new automation should target the Settings v2 paths exclusively.

Extensions credentials gained `EXTENSION_AUTHENTICATION` enum. The Credentials API (both Environment and Config) now supports an `EXTENSION_AUTHENTICATION` scope value. This enables extension-based authentication scenarios — for example, granting a stored credential the ability to authenticate Extensions API calls without expanding its broader scope.

Extensions Action endpoint signature change. `PUT /extensions/{extensionName}/monitoringConfigurations/{configurationId}/actions` now returns `agIds` (plural array) and deprecates the singular `agId` and `agName` properties. Update any code parsing the response to handle the array form.

> **Tokens:** For Extensions automation, prefer **Platform tokens (`dt0s16` / `dt0s01`)** with `Authorization: Bearer …` over the classic `dt0c01` (`Authorization: Api-Token …`). The Platform token model is the recommended path going forward — see AUTOM-08: Migration Automation for the migration plan.

---

2. API Fundamentals

Base URL Pattern

```
https://{tenant-id}.live.dynatrace.com/api/v2/settings
```

Authentication

All requests require an API token in the `Authorization` header:

```
Authorization: Api-Token &amp;lt;your-api-token&amp;gt;
```

Core Endpoints

Endpoint	Method	Purpose
`/api/v2/settings/schemas`	GET	List available schemas
`/api/v2/settings/schemas/{schemaId}`	GET	Get schema definition
`/api/v2/settings/objects`	GET	List/search objects
`/api/v2/settings/objects`	POST	Create objects
`/api/v2/settings/objects/{objectId}`	GET	Get specific object
`/api/v2/settings/objects/{objectId}`	PUT	Update object
`/api/v2/settings/objects/{objectId}`	DELETE	Delete object

Listing Available Schemas

To see all available configuration types:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
  -H "Authorization: Api-Token {token}" \
  -H "Accept: application/json"
```

Settings 2.0 Schema Catalog by Domain

This is the repo's consolidated Settings 2.0 schema catalog. **Corrected 07/01/2026:** the SLO row previously read `builtin:slo` — the actual schemaId, confirmed at the `dynatrace_slo_v2` Terraform resource docs, is `builtin:monitoring.slo`. That wrong value had propagated into the SLO series' own REFERENCE.md and SLO-05 before being fixed the same day; this table is now the single source other series should cite.

Reading the Upgrade status column

A schema being **available today** and a schema **surviving your upgrade to the latest Dynatrace** are two different questions, and several rows below answer them differently.

Question	What it means	Where it is answered
Is it deprecated?	Dynatrace has named a successor. An end-of-life date may or may not be published, and the schema keeps working meanwhile.	Product documentation, release notes
Is it blocked at upgrade?	The schema stops answering once the tenant moves to the latest Dynatrace.	The ready-made *Check your upgrade readiness* dashboard

The `Upgrade status` values below are read from that dashboard, observed **07/31/2026**. Note that public documentation does not currently publish these as breaking changes — a schema can read as perfectly current in the docs and still be flagged `Blocked` here. Blocking is triggered by **your tenant's own upgrade** rather than a calendar date, so the timing is yours; the scope is not. Re-check against the dashboard in your tenant before planning around any single row.

This matters most for config-as-code. Monaco resolves a schema before writing objects (`--settings-schema`), and the Terraform provider maps its resources onto the same schemas — so a `Blocked` row takes its Monaco config type and its Terraform resource with it. `/api/v2/settings/objects` itself is unaffected; it is the individual schema that goes away, not the endpoint.

Domain	Schema ID	Configuration type	Upgrade status
Classic config	`builtin:management-zones`	Management zones (see MZ2POL for the policy-migration path)	**Blocked**
Classic config	`builtin:tags.auto-tagging`	Auto-tagging rules	**Blocked**
Classic config	`builtin:tags.manual-tagging`	Manual tagging	**Blocked**
Classic config	`builtin:naming.hosts` / `.services` / `.processes-and-containers`	Conditional naming rules	**Blocked**
Alerting	`builtin:alerting.profile`	Alerting profiles	**Blocked** — successor is workflow-based notification (WFLOW, ALERT-03)
Alerting	`builtin:problem.notifications`	Problem notifications	**Blocked** — same successor
Alerting	`builtin:alerting.maintenance-window`	Maintenance windows	**Blocked** — successor is platform maintenance windows
Alerting	`builtin:anomaly-detection.hosts`	Host anomaly detection	Carries forward
Alerting	`builtin:anomaly-detection.services`	Service anomaly detection	Carries forward
Alerting	`builtin:anomaly-detection.metric-events`	Custom metric-event alerts (`dynatrace_metric_events` Terraform resource — not the singular, nonexistent `dynatrace_metric_event`)	**Blocked** — recreate as DQL-based detectors on `builtin:davis.anomaly-detectors`
Alerting	`builtin:anomaly-detection.infrastructure-disks`	Classic disk anomaly detection	**Blocked** — successor is the newer disk alerting
Alerting	`builtin:davis.anomaly-detectors`	DQL-based anomaly detectors	Carries forward — the Gen3 target for metric-event migrations
SLO	`builtin:monitoring.slo`	Service level objectives — **not** `builtin:slo`	**Blocked** — takes `dynatrace_slo_v2` and the Monaco path with it (SLO-05)
Service	`builtin:settings.calculated-service-metrics`	Calculated service metrics	**Blocked** — no successor *concept*, not just a renamed schema; each metric in use needs a replacement built on pipeline extraction

Domain	Schema ID	Configuration type	Upgrade status
Logs	`builtin:logmonitoring.log-custom-attributes` / `.log-dpp-rules` / `.log-buckets-rules` / `.log-events`	Classic log processing, attributes, buckets and events	**Blocked** — successor is OpenPipeline (OPMIG); do not point new automation at these
Cloud	`builtin:cloud.aws`	Classic AWS connection	**Blocked** — recreate as a Cloud Observability connection (CLOUD-02)
Infrastructure	`builtin:os.services.monitoring`	OS services monitoring rules	**Blocked**
Automation	`builtin:issue-tracking.integration`	Releases issue-tracking integrations	**Blocked** — the classic Releases app has no named successor; recreate in the new model
Network	`builtin:networkzones.zones`	Network zones — replaced the deprecated `/api/v2/networkZones` Configuration API (sprint 1.339); see M2S-03/04	**Verify** — the dashboard blocks `builtin:networkzones`, and prefix matching sweeps `.zones` in with it. The global toggle and the zone definitions are distinct schemas, so confirm which one is affected before acting
OpenPipeline	`builtin:openpipeline.<scope>.pipelines` / `.routing` / `.ingest-sources`	Per-data-type family, e.g. `builtin:openpipeline.logs.pipelines` — **not** a single generic schema; see OPIPE, OPMIG, SL2DT-03, NRLC-09	Carries forward
Service detection	`builtin:enhanced-endpoints-for-sdv1`	Service detection v1: auto-detect every endpoint and emit per-endpoint metrics (added v1.329; default on for environments created at v1.333+)	Carries forward — but must be **enabled** on every scope; scopes with it disabled break on upgrade

This table is not exhaustive of what is blocked. The readiness dashboard carries roughly 70 `builtin:` schema rules; the rows above are the ones this repo teaches or that readers commonly automate. Run the scan against your own tenant rather than treating an absence here as a clean bill of health. For the full migration sequence these rows sit inside, see the **Classic → Gen3 Platform** doorway in the `-START-HERE-` navigation playbook.

Not everything is a Settings 2.0 object. Two domains covered elsewhere in this repo deliberately have no row above because no Settings 2.0 schema exists for them: **FINOPS** (cost/budget configuration is driven by the Account Management API, not a Settings object) and **ONBRD** (OneAgent/ActiveGate deployment is driven by the Deployment API's installer endpoints, not a Settings object). If you're looking for a `builtin:` schema for either, stop — it doesn't exist; go to the API directly instead (see FINOPS/ONBRD REFERENCE.md for the specific endpoints).

---

3. Working with Settings Objects

Object Structure

Every settings object follows this structure:

```
{
  "schemaId": "builtin:management-zones",
  "scope": "environment",
  "value": {
    "name": "Production",
    "rules": [...]
  }
}
```

Field	Description
`schemaId`	The schema this object follows
`scope`	Where the config applies
`value`	The actual configuration data

Scope Types

Scope	Format	Example
Environment (tenant)	`environment`	`environment`
Host	`HOST-{id}`	`HOST-ABC123DEF456`
Host group	`HOST_GROUP-{id}`	`HOST_GROUP-XYZ789`
Service	`SERVICE-{id}`	`SERVICE-DEF456ABC`
Application	`APPLICATION-{id}`	`APPLICATION-123ABC`

Reading Objects

List all objects for a schema:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?schemaIds=builtin:management-zones"
\
  -H "Authorization: Api-Token {token}"
```

Get a specific object:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
  -H "Authorization: Api-Token {token}"
```

Query Parameters

Parameter	Description	Example
`schemaIds`	Filter by schema(s)	`schemaIds=builtin:management-zones`
`scopes`	Filter by scope(s)	`scopes=HOST-ABC123`
`fields`	Select specific fields	`fields=objectId,value`
`pageSize`	Results per page	`pageSize=100`

---

4. Common Operations

> **On the two examples below.** Management zones and auto-tagging are used here because they are the clearest illustrations of the request shape — a named object with a rule list — and because most existing tenants already have them. Both schemas are marked **Blocked** in the catalog above: they work today and stop answering when the tenant upgrades to the latest Dynatrace. Read them as *API mechanics*, not as configuration to go and create. For the replacements, see MZ2POL (management zones → policies and segments) and FAQ entry 02 (tagging strategy).

Create a Management Zone

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '[{
    "schemaId": "builtin:management-zones",
    "scope": "environment",
    "value": {
      "name": "Production-Web",
      "rules": [{
        "type": "SERVICE",
        "enabled": true,
        "conditions": [{
          "key": {
            "type": "STATIC",
            "attribute": "SERVICE_TAGS"
          },
          "comparisonInfo": {
            "type": "TAG",
            "operator": "TAG_KEY_EQUALS",
            "value": {
              "context": "CONTEXTLESS",
              "key": "environment"
            },
            "negate": false
          }
        }]
      }]
    }
  }]'
```

Create an Auto-Tagging Rule

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '[{
    "schemaId": "builtin:tags.auto-tagging",
    "scope": "environment",
    "value": {
      "name": "Application",
      "rules": [{
        "type": "SERVICE",
        "enabled": true,
        "valueFormat": "{Service:DetectedName}",
        "conditions": []
      }]
    }
  }]'
```

The request shape is identical for any schema that carries forward — swap the `schemaId` and the `value` block. `builtin:davis.anomaly-detectors` and the `builtin:openpipeline.<scope>.*` family are good next examples to try, and both survive the upgrade.

Update an Object

```
curl -X PUT "https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '{
    "value": {
      "name": "Production-Web-Updated",
      "rules": [...]
    }
  }'
```

Delete an Object

```
curl -X DELETE "https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
  -H "Authorization: Api-Token {token}"
```

Bulk Operations

Create multiple objects in one request:

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
  -H "Authorization: Api-Token {token}" \
  -H "Content-Type: application/json" \
  -d '[
    {"schemaId": "...", "scope": "...", "value": {...}},
    {"schemaId": "...", "scope": "...", "value": {...}},
    {"schemaId": "...", "scope": "...", "value": {...}}
  ]'
```

Error Handling

Common HTTP status codes:

Code	Meaning	Action
200	Success	Request completed
201	Created	Object created successfully
400	Bad Request	Check JSON syntax and schema
401	Unauthorized	Verify API token
403	Forbidden	Check token scopes
404	Not Found	Object or schema doesn't exist
409	Conflict	Object already exists
429	Rate Limited	Slow down requests

Validation Errors

The API returns detailed validation errors:

```
{
  "error": {
    "code": 400,
    "message": "Constraints violated",
    "constraintViolations": [
      {
        "path": "value.name",
        "message": "must not be blank"
      }
    ]
  }
}
```

---

5. Best Practices

API Token Security

Practice	Description
Least privilege	Only grant required scopes
Environment-specific	Separate tokens for dev/prod
Secret management	Use vaults, not hardcoded values
Rotation	Rotate tokens regularly

Rate Limiting

Limit Type	Recommendation
Requests per minute	Stay under 100 for bulk operations
Retry strategy	Exponential backoff on 429 errors
Batch operations	Use bulk endpoints where possible

Idempotency

Approach	Description
Check before create	Query for existing objects first
Use PUT for updates	PUT is idempotent, POST is not
Track object IDs	Store IDs for subsequent operations

Schema Discovery Pattern

Before creating objects, understand the schema:

1. **List schemas** to find the right one
2. **Get schema definition** for field requirements
3. **Review existing objects** for examples
4. **Validate** before bulk operations

```
# Step 1: Find the schema
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
  -H "Authorization: Api-Token {token}" | jq '.items[] | select(.schemaId | contains("management"))'

# Step 2: Get schema details
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas/builtin:management-zones" \
  -H "Authorization: Api-Token {token}"

# Step 3: See existing examples
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?schemaIds=builtin:management-zones" \
  -H "Authorization: Api-Token {token}"
```

---

6. Next Steps

When to Graduate from Settings API

Consider moving to higher-level tools when:

Scenario	Recommended Tool
Managing many configurations	Monaco
Need version control	Monaco or Terraform
Part of larger IaC pipeline	Terraform
Building a custom application	SDK

Continue the Series

Next Notebook	Focus
AUTOM-03: Monaco	Configuration-as-code with YAML

Additional Resources

- Settings API Documentation
- API Token Management
- Schema Reference

Summary

In this notebook, you learned:

- The Settings 2.0 API architecture and endpoints
- How to work with schemas, scopes, and objects

- Common CRUD operations for configuration
- Best practices for API token security and rate limiting

> **Key Takeaway:** The Settings API is the foundation for all Dynatrace configuration automation. Understanding it helps you debug issues with higher-level tools like Monaco and Terraform.

Continue to AUTOM-03: Monaco to learn configuration-as-code patterns.
